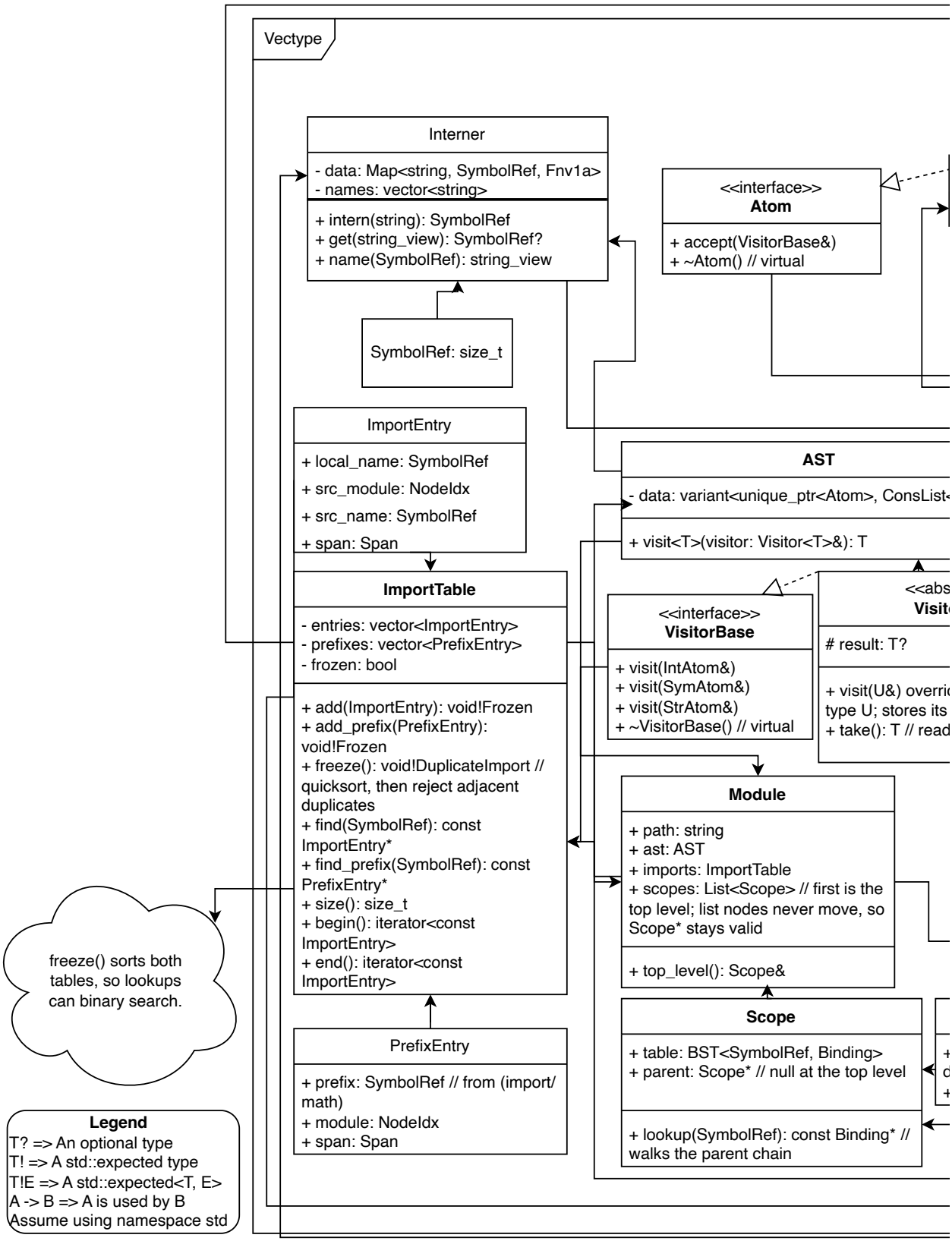
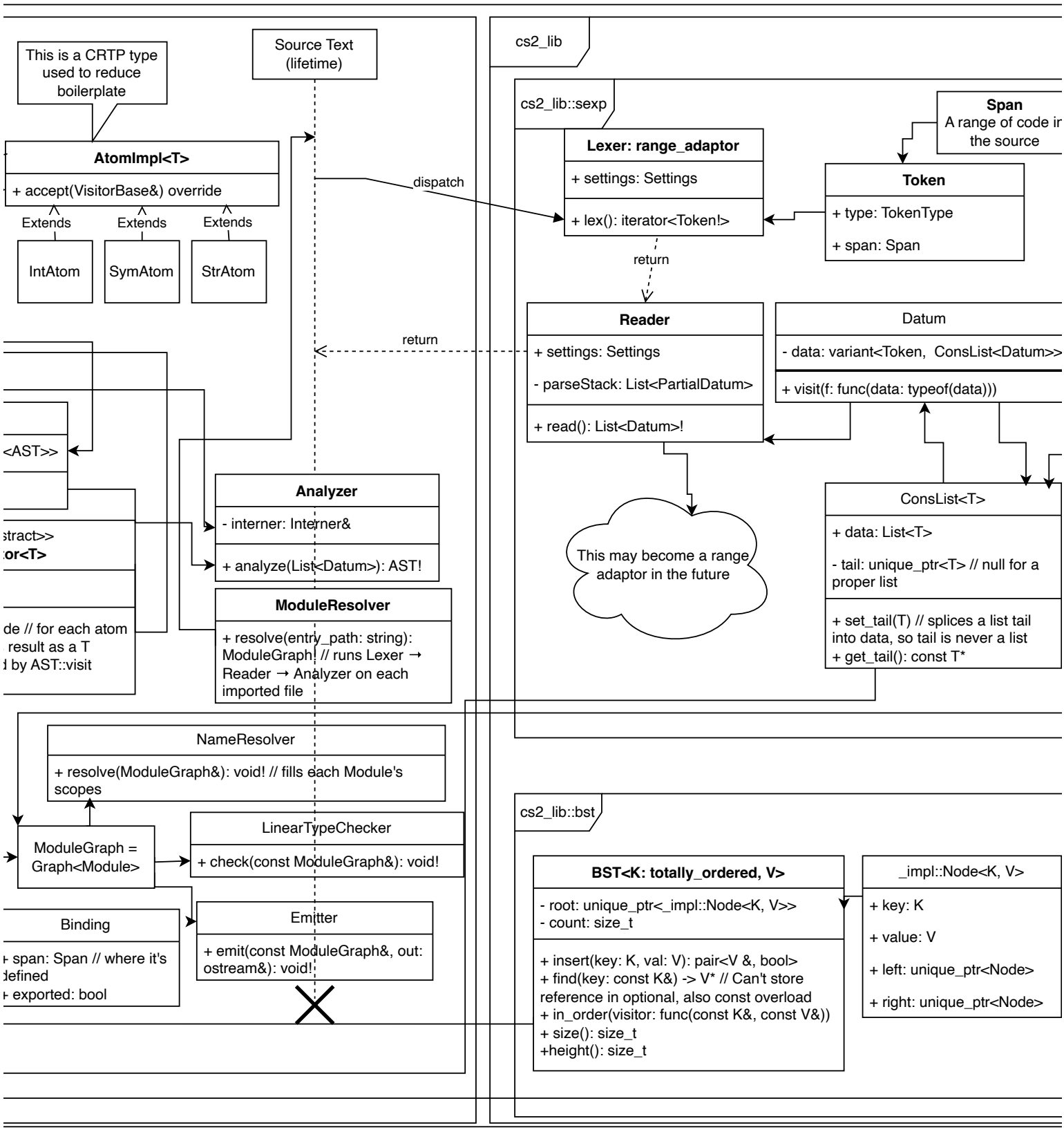
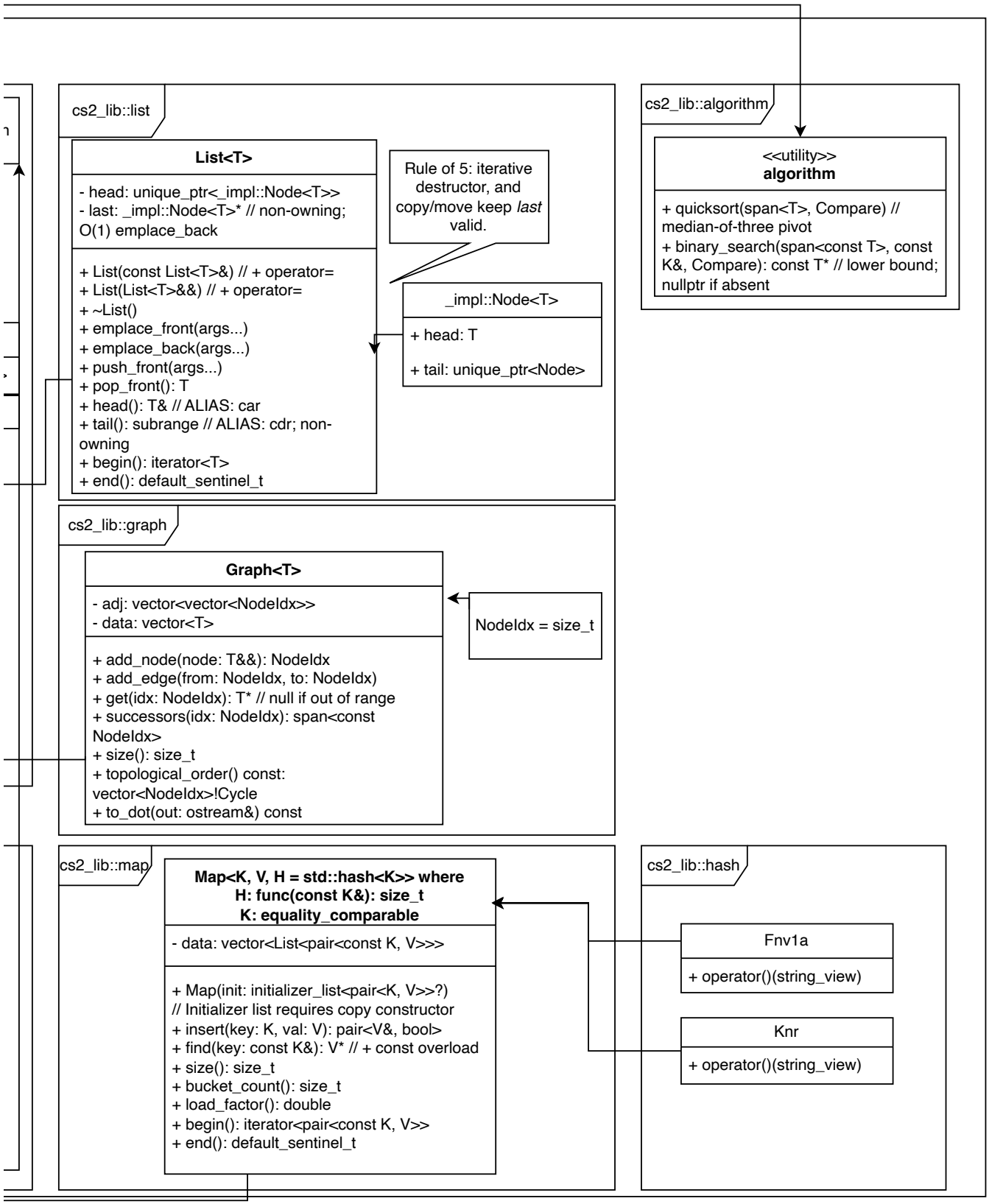

Vectype
Interner
- data: Map<string, SymbolRef, Fnv1a>
- names: vector<string>
+ intern(string): SymbolRef
+ get(string_view): SymbolRef?
+ name(SymbolRef): string_view
SymbolRef: size_t
<<interface>>
Atom
+ accept(VisitorBase&)
+ ~Atom() // virtual
ImportEntry
+ local_name: SymbolRef
+ src_module: NodeIdx
+ src_name: SymbolRef
+ span: Span
AST
- data: variant<unique_ptr<Atom>, ConsList<
+ visit<T>(visitor: Visitor<T>&): T
ImportTable
- entries: vector<ImportEntry>
- prefixes: vector<PrefixEntry>
- frozen: bool
+ add(ImportEntry): void!Frozen
+ add_prefix(PrefixEntry): void!Frozen
+ freeze(): void!DuplicateImport // quicksort, then reject adjacent duplicates
+ find(SymbolRef): const ImportEntry*
+ find_prefix(SymbolRef): const PrefixEntry*
+ size(): size_t
+ begin(): iterator<const ImportEntry>
+ end(): iterator<const ImportEntry>
<<interface>>
VisitorBase
+ visit(IntAtom&)
+ visit(SymAtom&)
+ visit(StrAtom&)
+ ~VisitorBase() // virtual
<<abs
Visit
result: T?
+ visit(U&) overri
type U; stores its
+ take(): T // read
Module
+ path: string
+ ast: AST
+ imports: ImportTable
+ scopes: List<Scope> // first is the top level; list nodes never move, so Scope* stays valid
+ top_level(): Scope&
freeze() sorts both tables, so lookups can binary search.
Scope
+ table: BST<SymbolRef, Binding>
+ parent: Scope* // null at the top level
+ lookup(SymbolRef): const Binding* // walks the parent chain
PrefixEntry
+ prefix: SymbolRef // from (import/math)
+ module: NodeIdx
+ span: Span
Legend
T? => An optional type
T! => A std::expected type
T!E => A std::expected<T, E>
A -> B => A is used by B
Assume using namespace std


This is a CRTP type used to reduce boilerplate
Source Text (lifetime)
cs2_lib
cs2_lib::sexp
AtomImpl<T>
+ accept(VisitorBase&) override
Extends
Extends
Extends
IntAtom
SymAtom
StrAtom
dispatch
Lexer: range_adaptor
+ settings: Settings
+ lex(): iterator<Token!>
Span
A range of code in the source
Token
+ type: TokenType
+ span: Span
return
Reader
+ settings: Settings
- parseStack: List<PartialDatum>
+ read(): List<Datum>!
return
Datum
- data: variant<Token, ConsList<Datum>>
+ visit(f: func(data: typeof(data)))
<AST>>
Analyzer
- interner: Interner&
+ analyze(List<Datum>): AST!
This may become a range adaptor in the future
ConsList<T>
+ data: List<T>
- tail: unique_ptr<T> // null for a proper list
+ set_tail(T) // splices a list tail into data, so tail is never a list
+ get_tail(): const T*
ModuleResolver
+ resolve(entry_path: string): ModuleGraph! // runs Lexer → Reader → Analyzer on each imported file
NameResolver
+ resolve(ModuleGraph&): void! // fills each Module's scopes
ModuleGraph = Graph<Module>
LinearTypeChecker
+ check(const ModuleGraph&): void!
Binding
+ span: Span // where it's defined
+ exported: bool
Emitter
+ emit(const ModuleGraph&, out: ostream&): void!
cs2_lib::bst
BST<K: totally_ordered, V>
- root: unique_ptr<_impl::Node<K, V>>
- count: size_t
+ insert(key: K, val: V): pair<V &, bool>
+ find(key: const K&) -> V* // Can't store reference in optional, also const overload
+ in_order(visitor: func(const K&, const V&))
+ size(): size_t
+height(): size_t
_impl::Node<K, V>
+ key: K
+ value: V
+ left: unique_ptr<Node>
+ right: unique_ptr<Node>


cs2_lib::list
List<T>
- head: unique_ptr<_impl::Node<T>>
- last: _impl::Node<T>* // non-owning; O(1) emplace_back
+ List(const List<T>&) // + operator=
+ List(List<T>&&) // + operator=
+ ~List()
+ emplace_front(args...)
+ emplace_back(args...)
+ push_front(args...)
+ pop_front(): T
+ head(): T& // ALIAS: car
+ tail(): subrange // ALIAS: cdr; non-owning
+ begin(): iterator<T>
+ end(): default_sentinel_t
Rule of 5: iterative destructor, and copy/move keep *last* valid.
_impl::Node<T>
+ head: T
+ tail: unique_ptr<Node>
cs2_lib::algorithm
<<utility>>
algorithm
+ quicksort(span<T>, Compare) // median-of-three pivot
+ binary_search(span<const T>, const K&, Compare): const T* // lower bound; nullptr if absent
cs2_lib::graph
Graph<T>
- adj: vector<vector<NodeIdx>>
- data: vector<T>
+ add_node(node: T&&): NodeIdx
+ add_edge(from: NodeIdx, to: NodeIdx)
+ get(idx: NodeIdx): T* // null if out of range
+ successors(idx: NodeIdx): span<const NodeIdx>
+ size(): size_t
+ topological_order() const: vector<NodeIdx>!Cycle
+ to_dot(out: ostream&) const
NodeIdx = size_t
cs2_lib::map
Map<K, V, H = std::hash<K>> where
H: func(const K&): size_t
K: equality_comparable
- data: vector<List<pair<const K, V>>>
+ Map(init: initializer_list<pair<K, V>>?)
// Initializer list requires copy constructor
+ insert(key: K, val: V): pair<V&, bool>
+ find(key: const K&): V* // + const overload
+ size(): size_t
+ bucket_count(): size_t
+ load_factor(): double
+ begin(): iterator<pair<const K, V>>
+ end(): default_sentinel_t
cs2_lib::hash
Fnv1a
+ operator()(string_view)
Knr
+ operator()(string_view)